Assignment — Implement Authentication & Authorization

Student Registration Web App

ASP.NET Core 8 MVC · Entity Framework Core · ASP.NET Core Identity

Name: Nikhil Ranjan Tripathi

Application No.: IN26009658

July 2026

Overview

The StudentRegistrationWebApp implements Authentication, Authorization, and Role-Based Navigation using ASP.NET Core Identity (Individual Accounts).

Roles: Admin (manages courses, views all students) and Student (views courses, registers for exactly one course, manages own profile). Every new registration is automatically assigned the Student role.

Administrator account (seeded at startup): admin@studentapp.com / Admin@123

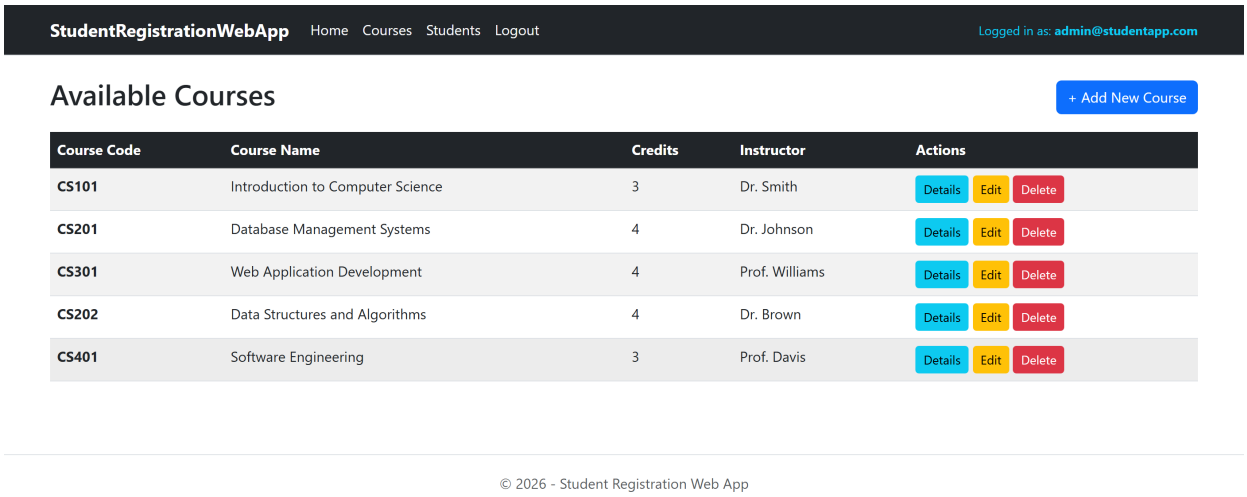
Students create their own accounts via the Register page. Five sample courses (CS101, CS201, CS301, CS202, CS401) are seeded on first run.

Security: [Authorize(Roles = ...)] attributes protect all restricted actions; typing a restricted URL directly redirects to Login (anonymous) or the Access Denied page (wrong role). A unique database index on CourseRegistrations.StudentId enforces the one-course-per-student rule at the database level. The navigation menu adapts to the visitor's role, and the logged-in user's email is always displayed in the navigation bar.

Development note: this is the second, improved iteration of the assignment. A first version was built on an Event Management domain (Admin/User roles, Events and Participants) using the default Identity UI. It was then redesigned as this Student Registration application to match the assignment specification exactly — with a custom Account controller, the Student role assigned at the moment of registration, profile fields on the Identity user itself, and the one-registration rule enforced by a unique database index rather than a code-only check.

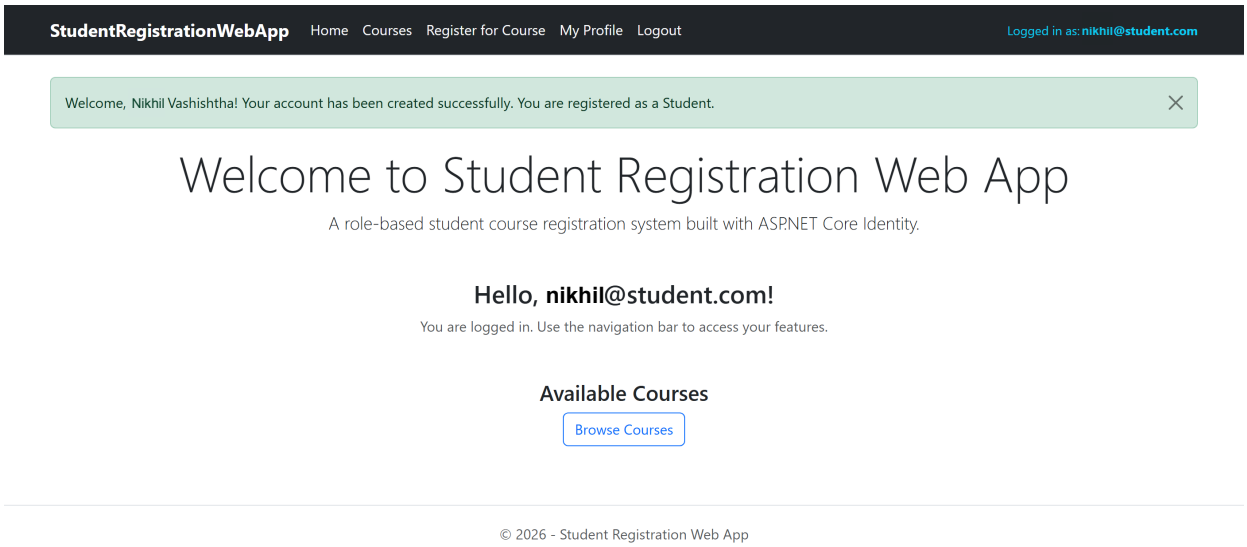
Part A — Screenshots

1. Administrator Navigation



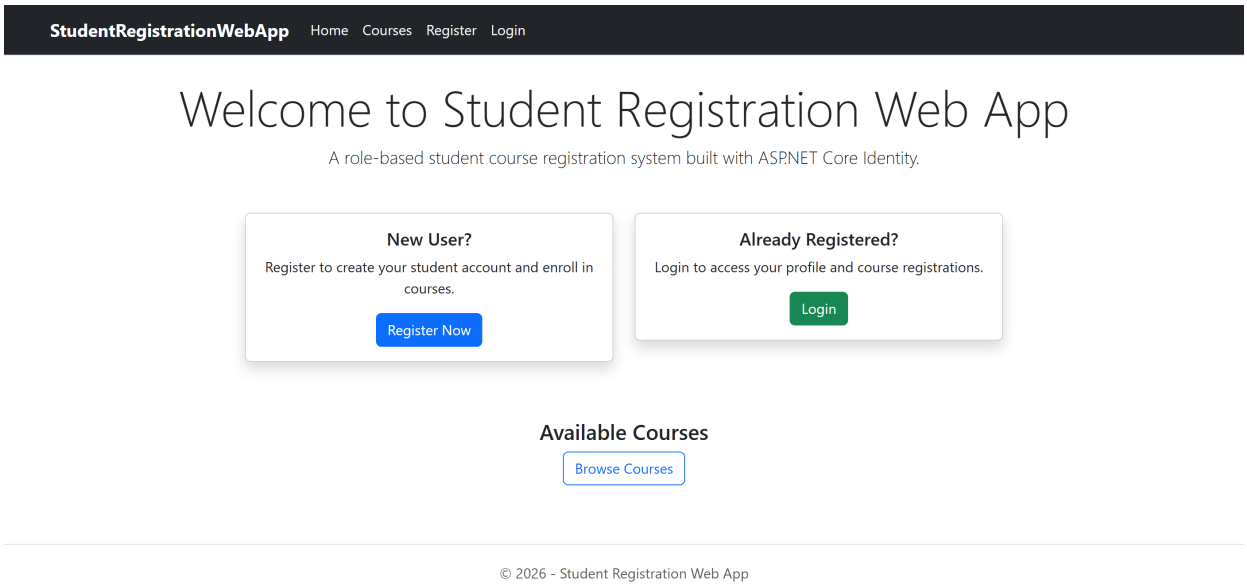
Admin sees: Home, Courses, Students, Logout — with the admin email displayed in the navigation bar. Add/Edit/Delete buttons are visible on the Courses list.

2. Student Navigation



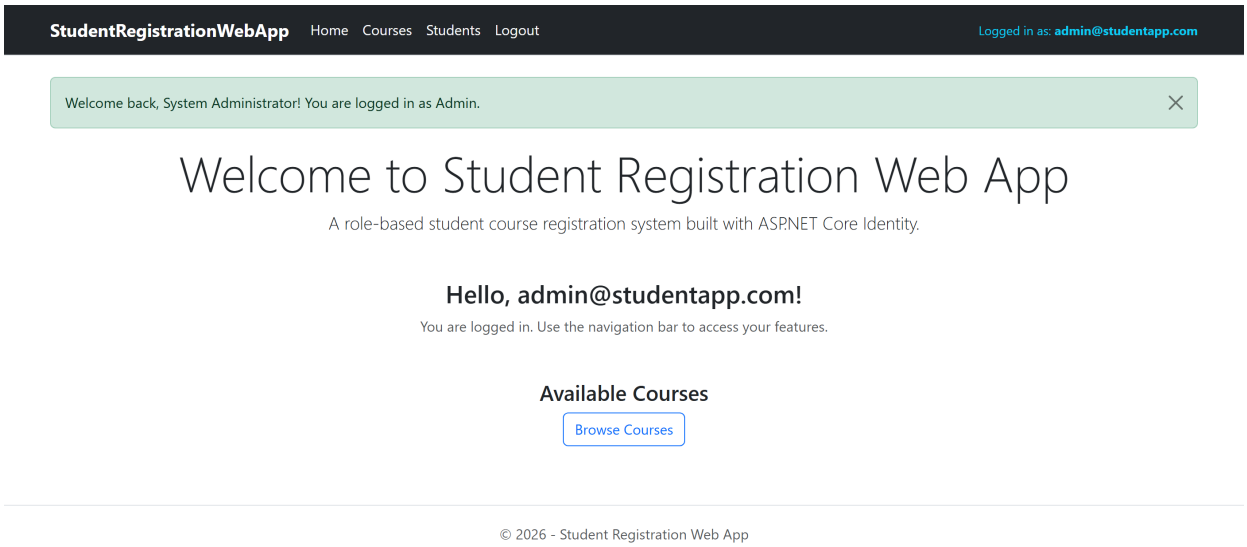
A newly registered student is automatically assigned the Student role and sees: Home, Courses, Register for Course, My Profile, Logout — with their email in the navigation bar. The welcome message confirms successful registration.

3. Anonymous Navigation



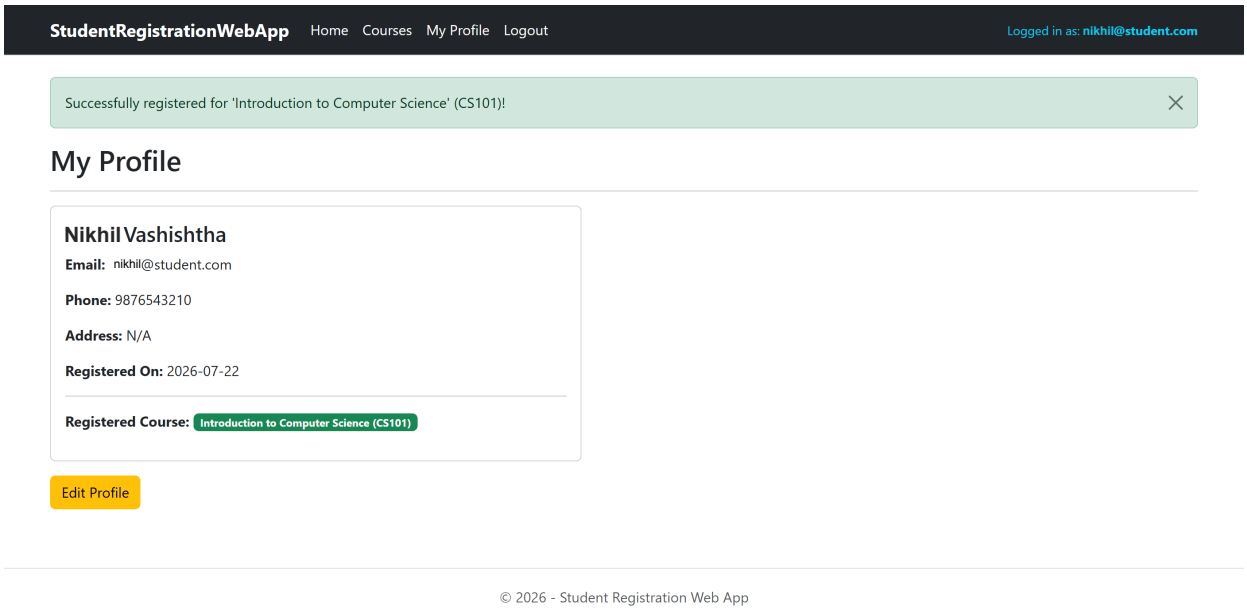
Anonymous visitors see only: Home, Courses, Register, Login.

4. Successful Login



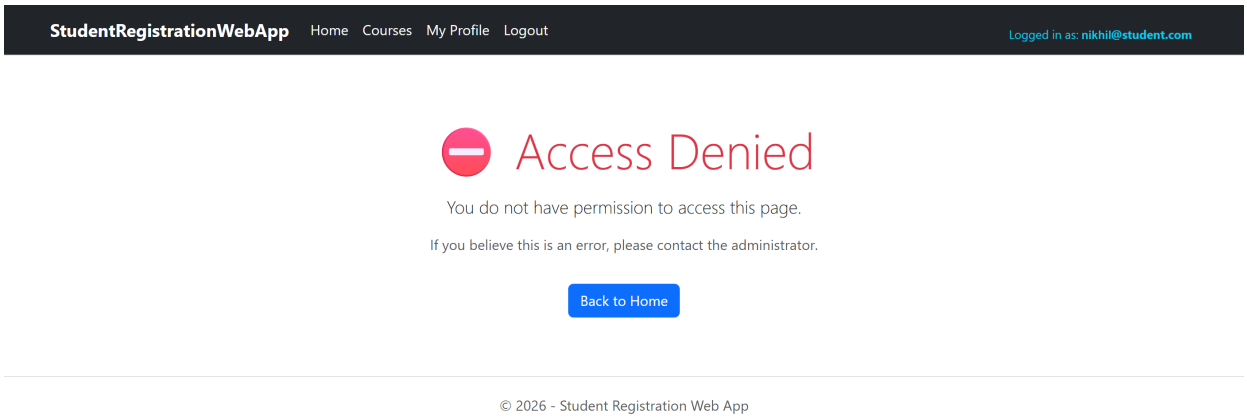
Welcome message displayed after a successful login, with the logged-in email shown in the navigation bar.

5. Successful Course Registration



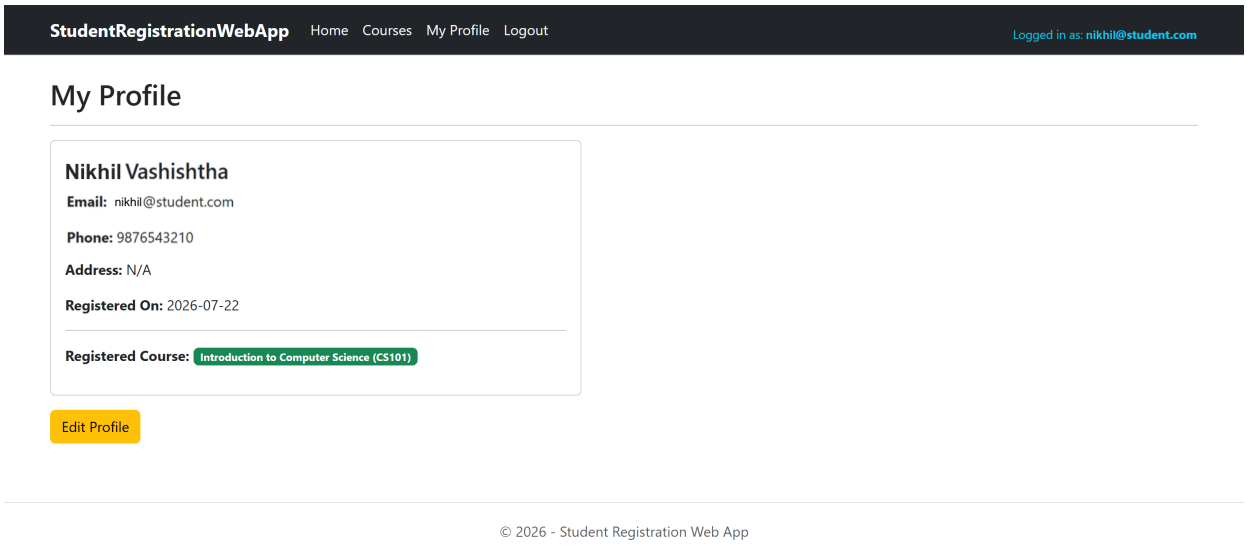
Success message after a student registers for a course. Note the 'Register for Course' link is no longer displayed in the navigation bar because the student has already registered for a course.

6. Access Denied Page



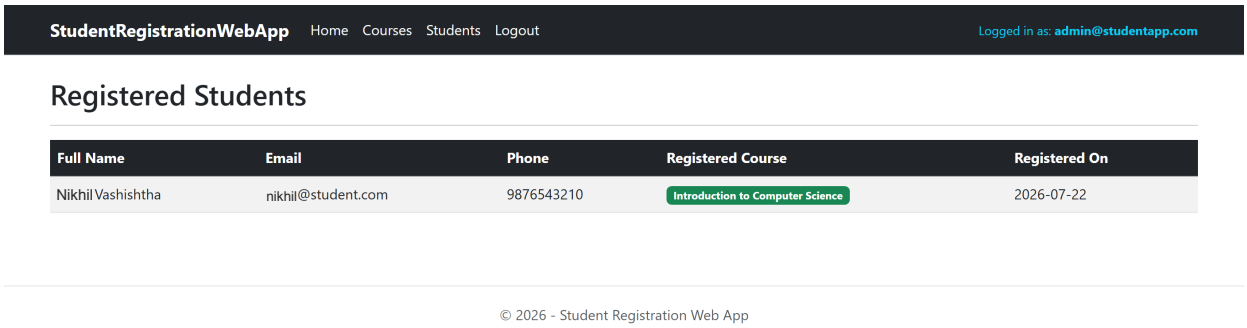
A logged-in Student typed the admin-only URL /Students directly and was shown the Access Denied page.

7. Student Profile Page



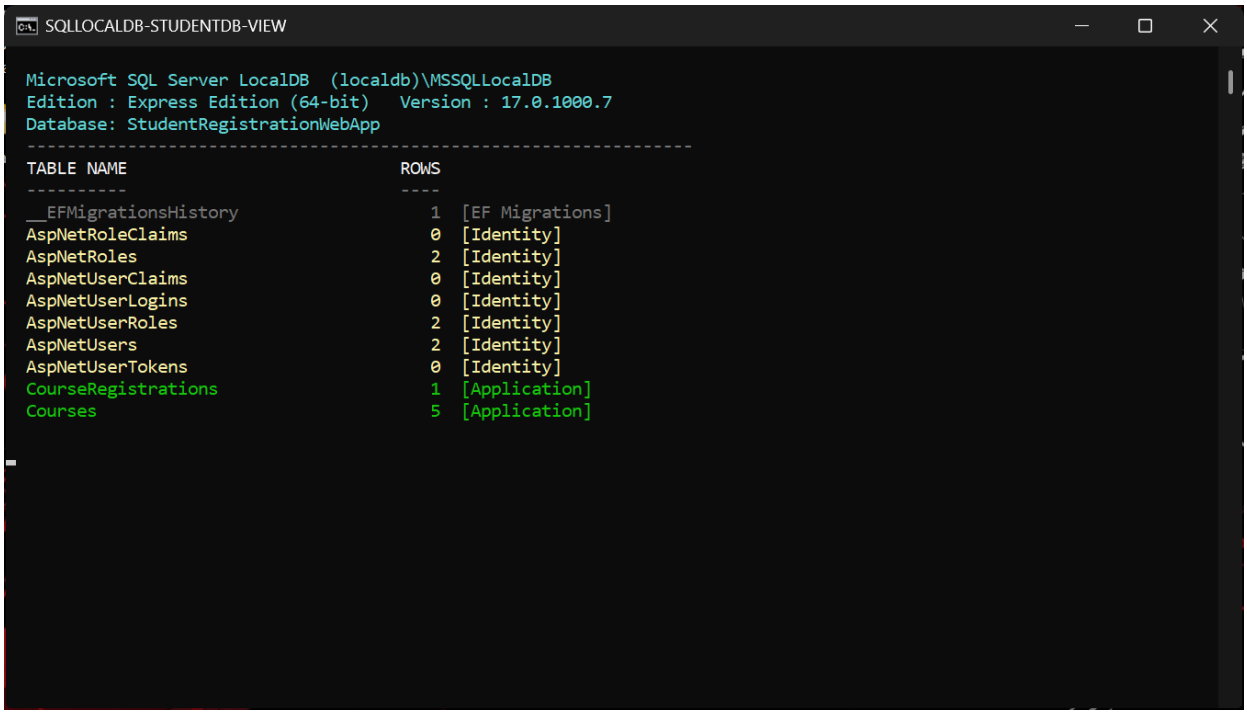
My Profile page showing the student's details and the course they registered for.

8. Administrator — List of Students



Admin-only view listing all registered students with their courses.

9. SQL Server Database — Identity and Application Tables



The StudentRegistrationWebApp database in SQL Server LocalDB, showing the ASP.NET Core Identity tables (AspNetUsers, AspNetRoles, AspNetUserRoles, ...) alongside the application tables (Courses, CourseRegistrations) with live row counts.

Part B — Complete Source Code

EF Core migrations (Migrations/) are generated code and are omitted here for brevity; they are included in the project repository: https://github.com/Nikhil-Ranjan-Tripathi/Student-registration-webapp

Project Configuration

StudentRegistrationWebApp.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <RootNamespace>StudentRegistrationWebApp</RootNamespace>
    <!-- v1-event-management is a separate reference project; keep it out of this build -->
    <DefaultItemExcludes>$(DefaultItemExcludes);v1-event-management\**</DefaultItemExcludes>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.Identity.EntityFrameworkCore" Version="8.0.8" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.8" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.8" />
    <PackageReference Include="Microsoft.VisualStudio.Web.CodeGeneration.Design" Version="8.0.4" />
  </ItemGroup>

</Project>
```

Program.cs

```
using StudentRegistrationWebApp.Data;
using StudentRegistrationWebApp.Models;
using Microsoft.AspNetCore.Identity;
```

```
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);

// ═══ Database ═══
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

// ═══ Identity ═══
builder.Services.AddIdentity<ApplicationUser, IdentityRole>(options =>
{
    options.Password.RequireDigit = true;
    options.Password.RequiredLength = 6;
    options.Password.RequireNonAlphanumeric = false;
    options.Password.RequireUppercase = false;
    options.User.RequireUniqueEmail = true;
})
.AddEntityFrameworkStores<ApplicationDbContext>()
.AddDefaultTokenProviders();

// Configure application cookie
builder.Services.ConfigureApplicationCookie(options =>
{
    options.LoginPath = "/Account/Login";
    options.LogoutPath = "/Account/Logout";
    options.AccessDeniedPath = "/Account/AccessDenied";
    options.ExpireTimeSpan = TimeSpan.FromHours(1);
});

builder.Services.AddControllersWithViews();
```

```
var app = builder.Build();


// ═══ Seed Database ═══
using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;
    try
    {
        var context = services.GetRequiredService<ApplicationDbContext>();
        var userManager = services.GetRequiredService<UserManager<ApplicationUser>>();
        var roleManager = services.GetRequiredService<RoleManager<IdentityRole>>();
        await DbInitializer.InitializeAsync(context, userManager, roleManager);
    }
    catch (Exception ex)
    {
        var logger = services.GetRequiredService<ILogger<Program>>();
        logger.LogError(ex, "An error occurred while seeding the database.");
    }
}


// ═══ Middleware ═══
if (app.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}
else
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}
```

```
app.UseHttpsRedirection();

app.UseStaticFiles();

app.UseRouting();

app.UseAuthentication();

app.UseAuthorization();


app.MapControllerRoute(

    name: "default",

    pattern: "{controller=Home}/{action=Index}/{id?}");


app.Run();
```

appsettings.json

```
{

  "ConnectionStrings": {

    "DefaultConnection": "Server=(localdb)\
\mssqllocaldb;Database=StudentRegistrationWebApp;Trusted_Connection=True;MultipleActiveResultSets=true"

  },

  "Logging": {

    "LogLevel": {

      "Default": "Information",

      "Microsoft.AspNetCore": "Warning"

    }

  },

  "AllowedHosts": "*"

}
```

Models

Models/ApplicationUser.cs

```
using Microsoft.AspNetCore.Identity;
```

```
namespace StudentRegistrationWebApp.Models
{
    public class ApplicationUser : IdentityUser
    {
        public string FullName { get; set; } = string.Empty;
        public string? Address { get; set; }
        public DateTime RegisteredOn { get; set; } = DateTime.Now;
    }
}
```

📄 Models/Course.cs

```
using System.ComponentModel.DataAnnotations;

namespace StudentRegistrationWebApp.Models
{
    public class Course
    {
        public int CourseId { get; set; }

        [Required(ErrorMessage = "Course name is required.")]
        [StringLength(100, ErrorMessage = "Course name cannot exceed 100 characters.")]
        [Display(Name = "Course Name")]
        public string CourseName { get; set; } = string.Empty;

        [Required(ErrorMessage = "Course code is required.")]
        [StringLength(20, ErrorMessage = "Course code cannot exceed 20 characters.")]
        [Display(Name = "Course Code")]
        public string CourseCode { get; set; } = string.Empty;

        [Required(ErrorMessage = "Description is required.")]
        [DataType(DataType.MultilineText)]
```

```
        public string Description { get; set; } = string.Empty;

        [Required(ErrorMessage = "Credits are required.")]
        [Range(1, 10, ErrorMessage = "Credits must be between 1 and 10.")]
        public int Credits { get; set; }

        [Display(Name = "Instructor")]
        public string Instructor { get; set; } = string.Empty;

        public ICollection<CourseRegistration>? Registrations { get; set; }
    }
}
```

Models/CourseRegistration.cs

```
using System.ComponentModel.DataAnnotations;

namespace StudentRegistrationWebApp.Models
{
    public class CourseRegistration
    {
        [Key]
        public int RegistrationId { get; set; }

        [Required]
        public string StudentId { get; set; } = string.Empty;

        [Required]
        public int CourseId { get; set; }

        [Display(Name = "Registration Date")]
        public DateTime RegistrationDate { get; set; } = DateTime.Now;
```

```
        // Navigation properties
        public ApplicationUser? Student { get; set; }
        public Course? Course { get; set; }
    }
}
```

Models/ErrorViewModel.cs

```
namespace StudentRegistrationWebApp.Models
{
    public class ErrorViewModel
    {
        public string? RequestId { get; set; }
        public bool ShowRequestId => !string.IsNullOrEmpty(RequestId);
    }
}
```

Data (DbContext + Seeder)

Data/ApplicationDbContext.cs

```
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;
using StudentRegistrationWebApp.Models;

namespace StudentRegistrationWebApp.Data
{
    public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
```

```
        }

        public DbSet<Course> Courses { get; set; }
        public DbSet<CourseRegistration> CourseRegistrations { get; set; }

        protected override void OnModelCreating(ModelBuilder builder)
        {
            base.OnModelCreating(builder);

            // Ensure a student can only register for ONE course
            builder.Entity<CourseRegistration>()
                .HasIndex(r => r.StudentId)
                .IsUnique();

            builder.Entity<CourseRegistration>()
                .HasOne(r => r.Student)
                .WithMany()
                .HasForeignKey(r => r.StudentId)
                .OnDelete(DeleteBehavior.Restrict);

            builder.Entity<CourseRegistration>()
                .HasOne(r => r.Course)
                .WithMany(c => c.Registrations)
                .HasForeignKey(r => r.CourseId)
                .OnDelete(DeleteBehavior.Cascade);
        }
    }
}
```

Data/DbInitializer.cs

```
using Microsoft.AspNetCore.Identity;
```

```
using Microsoft.EntityFrameworkCore;
using StudentRegistrationWebApp.Models;

namespace StudentRegistrationWebApp.Data
{
    public static class DbInitializer
    {
        public static async Task InitializeAsync(
            ApplicationDbContext context,
            UserManager<ApplicationUser> userManager,
            RoleManager<IdentityRole> roleManager)
        {
            // ═══ Ensure database is created ═══
            await context.Database.MigrateAsync();

            // ═══ Create Roles ═══
            string[] roles = { "Admin", "Student" };
            foreach (var role in roles)
            {
                if (!await roleManager.RoleExistsAsync(role))
                {
                    await roleManager.CreateAsync(new IdentityRole(role));
                }
            }

            // ═══ Create Administrator Account ═══
            string adminEmail = "admin@studentapp.com";
            string adminPassword = "Admin@123";

            var adminUser = await userManager.FindByEmailAsync(adminEmail);
            if (adminUser == null)
```

```
{
    adminUser = new ApplicationUser
    {
        UserName = adminEmail,
        Email = adminEmail,
        FullName = "System Administrator",
        EmailConfirmed = true,
        RegisteredOn = DateTime.Now
    };

    var result = await userManager.CreateAsync(adminUser, adminPassword);
    if (result.Succeeded)
    {
        await userManager.AddToRoleAsync(adminUser, "Admin");
    }
}
else if (!await userManager.IsInRoleAsync(adminUser, "Admin"))
{
    await userManager.AddToRoleAsync(adminUser, "Admin");
}

// ═══ Seed Sample Courses ═══
if (!context.Courses.Any())
{
    var courses = new Course[]
    {
        new Course
        {
            CourseName = "Introduction to Computer Science",
            CourseCode = "CS101",
            Description = "Fundamentals of computer science, algorithms, and programming.",
```

```
        Credits = 3,
        Instructor = "Dr. Smith"
    },
    new Course
    {
        CourseName = "Database Management Systems",
        CourseCode = "CS201",
        Description = "Relational databases, SQL, normalization, and transaction management.",
        Credits = 4,
        Instructor = "Dr. Johnson"
    },
    new Course
    {
        CourseName = "Web Application Development",
        CourseCode = "CS301",
        Description = "ASP.NET Core MVC, Entity Framework, and modern web development.",
        Credits = 4,
        Instructor = "Prof. Williams"
    },
    new Course
    {
        CourseName = "Data Structures and Algorithms",
        CourseCode = "CS202",
        Description = "Trees, graphs, sorting, searching, and algorithm analysis.",
        Credits = 4,
        Instructor = "Dr. Brown"
    },
    new Course
    {
        CourseName = "Software Engineering",
        CourseCode = "CS401",
```

```
                        Description = "Software development lifecycle, design patterns, and project
management.",
                        Credits = 3,
                        Instructor = "Prof. Davis"
                    }
                };

                context.Courses.AddRange(courses);
                await context.SaveChangesAsync();
            }
        }
    }
}
```

Controllers

Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
using StudentRegistrationWebApp.Models;

namespace StudentRegistrationWebApp.Controllers
{
    public class HomeController : Controller
    {
        private readonly ILogger<HomeController> _logger;

        public HomeController(ILogger<HomeController> logger)
        {
            _logger = logger;
        }

        // ═══ Anonymous users can view Home ═══
```

```
        public IActionResult Index()
        {
            return View();
        }

        [ResponseCache(Duration = 0, Location = ResponseCacheLocation.None, NoStore = true)]
        public IActionResult Error()
        {
            return View(new ErrorViewModel { RequestId = HttpContext.TraceIdentifier });
        }
    }
}
```

Controllers/AccountController.cs

```
using System.ComponentModel.DataAnnotations;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using StudentRegistrationWebApp.Data;
using StudentRegistrationWebApp.Models;

namespace StudentRegistrationWebApp.Controllers
{
    // ═══ Account Controller — Login, Register, Logout, Access Denied ═══
    public class AccountController : Controller
    {
        private readonly UserManager<ApplicationUser> _userManager;
        private readonly SignInManager<ApplicationUser> _signInManager;
        private readonly RoleManager<IdentityRole> _roleManager;
        private readonly ApplicationDbContext _context;
```

```
public AccountController(
    UserManager<ApplicationUser> userManager,
    SignInManager<ApplicationUser> signInManager,
    RoleManager<IdentityRole> roleManager,
    ApplicationDbContext context)
{
    _userManager = userManager;
    _signInManager = signInManager;
    _roleManager = roleManager;
    _context = context;
}

// ═══ GET: Account/Register ═══
[HttpGet]
[AllowAnonymous]
public IActionResult Register()
{
    return View();
}

// ═══ POST: Account/Register ═══
[HttpPost]
[ValidateAntiForgeryToken]
[AllowAnonymous]
public async Task<IActionResult> Register(RegisterViewModel model)
{
    if (ModelState.IsValid)
    {
        var user = new ApplicationUser
        {
```

```
                UserName = model.Email,
                Email = model.Email,
                FullName = model.FullName,
                PhoneNumber = model.PhoneNumber ?? "",
                EmailConfirmed = true,
                RegisteredOn = DateTime.Now
            };

            var result = await _userManager.CreateAsync(user, model.Password);

            if (result.Succeeded)
            {
                // ═══ Auto-assign "Student" role to every new user ═══
                await _userManager.AddToRoleAsync(user, "Student");

                await _signInManager.SignInAsync(user, isPersistent: false);
                TempData["SuccessMessage"] = $"Welcome, {user.FullName}! Your account has been created
successfully. You are registered as a Student.";
                return RedirectToAction("Index", "Home");
            }

            foreach (var error in result.Errors)
            {
                ModelState.AddModelError(string.Empty, error.Description);
            }
        }
        return View(model);
    }

    // ═══ GET: Account/Login ═══
    [HttpGet]
    [AllowAnonymous]
```

```
        public IActionResult Login(string? returnUrl = null)
        {
            ViewData["ReturnUrl"] = returnUrl;
            return View();
        }

        // ═══ POST: Account/Login ═══
        [HttpPost]
        [ValidateAntiForgeryToken]
        [AllowAnonymous]
        public async Task<IActionResult> Login(LoginViewModel model, string? returnUrl = null)
        {
            ViewData["ReturnUrl"] = returnUrl;

            if (ModelState.IsValid)
            {
                var result = await _signInManager.PasswordSignInAsync(
                    model.Email, model.Password, model.RememberMe, lockoutOnFailure: false);

                if (result.Succeeded)
                {
                    var user = await _userManager.FindByEmailAsync(model.Email);
                    if (user != null)
                    {
                        var roles = await _userManager.GetRolesAsync(user);
                        var roleStr = roles.Count > 0 ? roles[0] : "User";
                        TempData["SuccessMessage"] = $"Welcome back, {user.FullName}! You are logged in as
{roleStr}.";
                    }

                    if (!string.IsNullOrEmpty(returnUrl) && Url.IsLocalUrl(returnUrl))
                        return Redirect(returnUrl);
```

```
                return RedirectToAction("Index", "Home");
            }

            ModelState.AddModelError(string.Empty, "Invalid login attempt. Check your email and password.");
        }
        return View(model);
    }

    // === POST: Account/Logout ===
    [HttpPost]
    [ValidateAntiForgeryToken]
    [Authorize]
    public async Task<IActionResult> Logout()
    {
        await _signInManager.SignOutAsync();
        TempData["SuccessMessage"] = "You have been logged out successfully.";
        return RedirectToAction("Index", "Home");
    }

    // === GET: Account/AccessDenied ===
    [HttpGet]
    [AllowAnonymous]
    public IActionResult AccessDenied()
    {
        return View();
    }
}

// === View Models ===
public class RegisterViewModel
```

```
{
    [Required(ErrorMessage = "Full name is required.")]
    [Display(Name = "Full Name")]
    public string FullName { get; set; } = string.Empty;

    [Required(ErrorMessage = "Email is required.")]
    [EmailAddress(ErrorMessage = "Invalid email address.")]
    public string Email { get; set; } = string.Empty;

    [Required(ErrorMessage = "Password is required.")]
    [DataType(DataType.Password)]
    [StringLength(100, ErrorMessage = "Password must be at least {2} characters.", MinimumLength = 6)]
    public string Password { get; set; } = string.Empty;

    [Required(ErrorMessage = "Please confirm your password.")]
    [DataType(DataType.Password)]
    [Display(Name = "Confirm Password")]
    [Compare("Password", ErrorMessage = "Passwords do not match.")]
    public string ConfirmPassword { get; set; } = string.Empty;

    [Display(Name = "Phone Number")]
    [Phone(ErrorMessage = "Invalid phone number.")]
    public string? PhoneNumber { get; set; }
}

public class LoginViewModel
{
    [Required(ErrorMessage = "Email is required.")]
    [EmailAddress(ErrorMessage = "Invalid email address.")]
    public string Email { get; set; } = string.Empty;
```

```
        [Required(ErrorMessage = "Password is required.")]
        [DataType(DataType.Password)]
        public string Password { get; set; } = string.Empty;

        [Display(Name = "Remember me?")]
        public bool RememberMe { get; set; }
    }
}
```

Controllers/CoursesController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using StudentRegistrationWebApp.Data;
using StudentRegistrationWebApp.Models;

namespace StudentRegistrationWebApp.Controllers
{
    public class CoursesController : Controller
    {
        private readonly ApplicationDbContext _context;

        public CoursesController(ApplicationDbContext context)
        {
            _context = context;
        }

        // ═══ GET: Courses — Anonymous users can view course list ═══
        [AllowAnonymous]
        public async Task<IActionResult> Index()
        {
```

```
            var courses = await _context.Courses.ToListAsync();

            return View(courses);

        }


        // ═══ GET: Courses/Details/5 — Anonymous users can view details ═══

        [AllowAnonymous]

        public async Task<IActionResult> Details(int? id)

        {

            if (id == null) return NotFound();


            var course = await _context.Courses

                .FirstOrDefaultAsync(c => c.CourseId == id);


            if (course == null) return NotFound();


            return View(course);

        }


        // ═══ GET: Courses/Create — Admin only ═══

        [Authorize(Roles = "Admin")]

        public IActionResult Create()

        {

            return View();

        }


        // ═══ POST: Courses/Create — Admin only ═══

        [HttpPost]

        [ValidateAntiForgeryToken]

        [Authorize(Roles = "Admin")]

        public async Task<IActionResult>
Create([Bind("CourseId,CourseName,CourseCode,Description,Credits,Instructor")] Course course)

        {
```

```
            if (ModelState.IsValid)
            {
                _context.Add(course);
                await _context.SaveChangesAsync();
                TempData["SuccessMessage"] = $"Course '{course.CourseName}' created successfully!";
                return RedirectToAction(nameof(Index));
            }
            return View(course);
        }

        // === GET: Courses/Edit/5 – Admin only ===
        [Authorize(Roles = "Admin")]
        public async Task<IActionResult> Edit(int? id)
        {
            if (id == null) return NotFound();

            var course = await _context.Courses.FindAsync(id);
            if (course == null) return NotFound();

            return View(course);
        }

        // === POST: Courses/Edit/5 – Admin only ===
        [HttpPost]
        [ValidateAntiForgeryToken]
        [Authorize(Roles = "Admin")]
        public async Task<IActionResult> Edit(int id,
[Bind("CourseId,CourseName,CourseCode,Description,Credits,Instructor")] Course course)
        {
            if (id != course.CourseId) return NotFound();

            if (ModelState.IsValid)
```

```
    {
        try
        {
            _context.Update(course);
            await _context.SaveChangesAsync();
            TempData["SuccessMessage"] = $"Course '{course.CourseName}' updated successfully!";
        }
        catch (DbUpdateConcurrencyException)
        {
            if (!CourseExists(course.CourseId)) return NotFound();
            else throw;
        }
        return RedirectToAction(nameof(Index));
    }
    return View(course);
}

// ═══ GET: Courses/Delete/5 — Admin only ═══
[Authorize(Roles = "Admin")]
public async Task<IActionResult> Delete(int? id)
{
    if (id == null) return NotFound();

    var course = await _context.Courses
        .FirstOrDefaultAsync(c => c.CourseId == id);

    if (course == null) return NotFound();

    return View(course);
}
```

```
        // ═══ POST: Courses/Delete/5 — Admin only ═══
        [HttpPost, ActionName("Delete")]
        [ValidateAntiForgeryToken]
        [Authorize(Roles = "Admin")]
        public async Task<IActionResult> DeleteConfirmed(int id)
        {
            var course = await _context.Courses.FindAsync(id);
            if (course != null)
            {
                // Also delete registrations for this course
                var registrations = _context.CourseRegistrations.Where(r => r.CourseId == id);
                _context.CourseRegistrations.RemoveRange(registrations);

                _context.Courses.Remove(course);
                await _context.SaveChangesAsync();
                TempData["SuccessMessage"] = $"Course '{course.CourseName}' deleted successfully!";
            }
            return RedirectToAction(nameof(Index));
        }

        private bool CourseExists(int id)
        {
            return _context.Courses.Any(e => e.CourseId == id);
        }
    }
}
```

Controllers/StudentsController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
```

```
using Microsoft.EntityFrameworkCore;
using StudentRegistrationWebApp.Data;
using StudentRegistrationWebApp.Models;

namespace StudentRegistrationWebApp.Controllers
{
    // ═══ Admin only — View all registered students ═══
    [Authorize(Roles = "Admin")]
    public class StudentsController : Controller
    {
        private readonly ApplicationDbContext _context;
        private readonly UserManager<ApplicationUser> _userManager;

        public StudentsController(ApplicationDbContext context, UserManager<ApplicationUser> userManager)
        {
            _context = context;
            _userManager = userManager;
        }

        // GET: Students — Admin only
        public async Task<IActionResult> Index()
        {
            // Get all users in Student role
            var students = await _userManager.GetUsersInRoleAsync("Student");

            // Include their course registrations
            var studentIds = students.Select(s => s.Id).ToList();
            var registrations = await _context.CourseRegistrations
                .Include(r => r.Course)
                .Where(r => studentIds.Contains(r.StudentId))
                .ToListAsync();
```

```
            var studentList = students.Select(s => new StudentViewModel

            {

                Id = s.Id,

                FullName = s.FullName,

                Email = s.Email ?? "",

                PhoneNumber = s.PhoneNumber ?? "",

                RegisteredOn = s.RegisteredOn,

                RegisteredCourse = registrations.FirstOrDefault(r => r.StudentId ==
s.Id)?.Course?.CourseName ?? "Not Registered"

            }).ToList();



            return View(studentList);

        }

    }



    public class StudentViewModel

    {

        public string Id { get; set; } = string.Empty;

        public string FullName { get; set; } = string.Empty;

        public string Email { get; set; } = string.Empty;

        public string PhoneNumber { get; set; } = string.Empty;

        public DateTime RegisteredOn { get; set; }

        public string RegisteredCourse { get; set; } = string.Empty;

    }

}
```

Controllers/ProfileController.cs

```
using System.ComponentModel.DataAnnotations;

using Microsoft.AspNetCore.Authorization;

using Microsoft.AspNetCore.Identity;
```

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using StudentRegistrationWebApp.Data;
using StudentRegistrationWebApp.Models;

namespace StudentRegistrationWebApp.Controllers
{
    // ═══ Student only — View and edit own profile ═══
    [Authorize(Roles = "Student")]
    public class ProfileController : Controller
    {
        private readonly UserManager<ApplicationUser> _userManager;
        private readonly ApplicationDbContext _context;

        public ProfileController(UserManager<ApplicationUser> userManager, ApplicationDbContext context)
        {
            _userManager = userManager;
            _context = context;
        }

        // GET: Profile — Student views their own profile
        public async Task<IActionResult> Index()
        {
            var user = await _userManager.GetUserAsync(User);
            if (user == null) return NotFound();

            // Check if student has registered for a course
            var registration = await _context.CourseRegistrations
                .Include(r => r.Course)
                .FirstOrDefaultAsync(r => r.StudentId == user.Id);
```

```
    var model = new ProfileViewModel
    {
        Id = user.Id,
        FullName = user.FullName,
        Email = user.Email ?? "",
        PhoneNumber = user.PhoneNumber ?? "",
        Address = user.Address ?? "",
        RegisteredOn = user.RegisteredOn,
        RegisteredCourse = registration?.Course?.CourseName ?? "Not Registered",
        CourseCode = registration?.Course?.CourseCode ?? ""
    };

    return View(model);
}

// GET: Profile/Edit — Student edits their own profile
public async Task<IActionResult> Edit()
{
    var user = await _userManager.GetUserAsync(User);
    if (user == null) return NotFound();

    var model = new EditProfileViewModel
    {
        FullName = user.FullName,
        PhoneNumber = user.PhoneNumber ?? "",
        Address = user.Address ?? ""
    };

    return View(model);
}
```

```
    // POST: Profile/Edit
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Edit(EditProfileViewModel model)
    {
        if (!ModelState.IsValid) return View(model);

        var user = await _userManager.GetUserAsync(User);
        if (user == null) return NotFound();

        user.FullName = model.FullName;
        user.PhoneNumber = model.PhoneNumber;
        user.Address = model.Address;

        var result = await _userManager.UpdateAsync(user);
        if (result.Succeeded)
        {
            TempData["SuccessMessage"] = "Profile updated successfully!";
            return RedirectToAction(nameof(Index));
        }

        foreach (var error in result.Errors)
        {
            ModelState.AddModelError(string.Empty, error.Description);
        }

        return View(model);
    }
}

public class ProfileViewModel
```

```
    {
        public string Id { get; set; } = string.Empty;
        public string FullName { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
        public string PhoneNumber { get; set; } = string.Empty;
        public string Address { get; set; } = string.Empty;
        public DateTime RegisteredOn { get; set; }
        public string RegisteredCourse { get; set; } = string.Empty;
        public string CourseCode { get; set; } = string.Empty;
    }

    public class EditProfileViewModel
    {
        [Required(ErrorMessage = "Full name is required.")]
        [Display(Name = "Full Name")]
        public string FullName { get; set; } = string.Empty;

        [Display(Name = "Phone Number")]
        [Phone(ErrorMessage = "Invalid phone number.")]
        public string PhoneNumber { get; set; } = string.Empty;

        [Display(Name = "Address")]
        public string Address { get; set; } = string.Empty;
    }
}
```

Controllers/CourseRegistrationController.cs

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
```

```
using StudentRegistrationWebApp.Data;
using StudentRegistrationWebApp.Models;

namespace StudentRegistrationWebApp.Controllers
{
    // ═══ Course Registration — Student can register for ONE course only ═══
    [Authorize(Roles = "Student")]
    public class CourseRegistrationController : Controller
    {
        private readonly ApplicationDbContext _context;
        private readonly UserManager<ApplicationUser> _userManager;

        public CourseRegistrationController(ApplicationDbContext context, UserManager<ApplicationUser>
userManager)
        {
            _context = context;
            _userManager = userManager;
        }

        // GET: CourseRegistration/Register — Show available courses for registration
        public async Task<IActionResult> Register()
        {
            var user = await _userManager.GetUserAsync(User);
            if (user == null) return NotFound();

            // Check if student already registered for a course
            var existingRegistration = await _context.CourseRegistrations
                .Include(r => r.Course)
                .FirstOrDefaultAsync(r => r.StudentId == user.Id);

            if (existingRegistration != null)
            {
```

```
                TempData["InfoMessage"] = $"You have already registered for
'{existingRegistration.Course?.CourseName}'. Students can register for only one course.";

                return RedirectToAction("Index", "Profile");

            }



            // Show available courses

            var courses = await _context.Courses.ToListAsync();

            return View(courses);

        }



        // POST: CourseRegistration/Register/5 — Register for a specific course

        [HttpPost]

        [ValidateAntiForgeryToken]

        public async Task<IActionResult> Register(int courseId)

        {

            var user = await _userManager.GetUserAsync(User);

            if (user == null) return NotFound();



            // Double-check student hasn't already registered

            var existing = await _context.CourseRegistrations

                .FirstOrDefaultAsync(r => r.StudentId == user.Id);



            if (existing != null)

            {

                TempData["ErrorMessage"] = "You have already registered for a course. Students can only
register for one course.";

                return RedirectToAction("Index", "Profile");

            }



            var course = await _context.Courses.FindAsync(courseId);

            if (course == null)

            {
```

```
            TempData["ErrorMessage"] = "Course not found.";
            return RedirectToAction(nameof(Register));
        }

        var registration = new CourseRegistration
        {
            StudentId = user.Id,
            CourseId = courseId,
            RegistrationDate = DateTime.Now
        };

        _context.CourseRegistrations.Add(registration);
        await _context.SaveChangesAsync();

        TempData["SuccessMessage"] = $"Successfully registered for '{course.CourseName}'
({course.CourseCode})!";
        return RedirectToAction("Index", "Profile");
    }

    // POST: CourseRegistration/Drop – Drop the registered course
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Drop()
    {
        var user = await _userManager.GetUserAsync(User);
        if (user == null) return NotFound();

        var registration = await _context.CourseRegistrations
            .FirstOrDefaultAsync(r => r.StudentId == user.Id);

        if (registration != null)
        {
```

```
                var course = await _context.Courses.FindAsync(registration.CourseId);

                _context.CourseRegistrations.Remove(registration);

                await _context.SaveChangesAsync();

                TempData["SuccessMessage"] = $"Dropped course '{course?.CourseName}'. You can now register for
a different course.";

            }



            return RedirectToAction(nameof(Register));

        }

    }

}
```

Views — Shared

Views/_ViewImports.cshtml

```
@using StudentRegistrationWebApp

@using StudentRegistrationWebApp.Models

@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

Views/_ViewStart.cshtml

```
@{

    Layout = "_Layout";

}
```

Views/Shared/_Layout.cshtml

```
@using Microsoft.AspNetCore.Identity

@using Microsoft.EntityFrameworkCore

@using StudentRegistrationWebApp.Data

@using StudentRegistrationWebApp.Models

@inject SignInManager<ApplicationUser> SignInManager

@inject UserManager<ApplicationUser> UserManager
```

```
@inject ApplicationDbContext DbContext

@{
    var user = SignInManager.IsSignedIn(User) ? await UserManager.GetUserAsync(User) : null;
    var isAdmin = user != null && await UserManager.IsInRoleAsync(user, "Admin");
    var isStudent = user != null && await UserManager.IsInRoleAsync(user, "Student");
    var hasRegisteredCourse = isStudent && user != null
        && await DbContext.CourseRegistrations.AnyAsync(r => r.StudentId == user.Id);
}

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>@ViewData["Title"] - Student Registration Web App</title>
    <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
</head>
<body>
    <header>
        <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-dark bg-dark border-bottom box-shadow">
            <div class="container">
                <a class="navbar-brand" asp-controller="Home" asp-action="Index">StudentRegistrationWebApp</a>
                <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target=".navbar-
collapse">
                    <span class="navbar-toggler-icon"></span>
                </button>
                <div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
                    <ul class="navbar-nav flex-grow-1">
                        <li class="nav-item">
                            <a class="nav-link text-light" asp-controller="Home" asp-action="Index">Home</a>
```

```
                        </li>

                        <!-- ═══ Courses — visible to all (anonymous + authenticated) ═══ -->
                        <li class="nav-item">
                            <a class="nav-link text-light" asp-controller="Courses" asp-
action="Index">Courses</a>
                        </li>

                        <!-- ═══ Register for Course — Student only, if not already registered ═══ -->
                        @if (isStudent)
                        {
                            @if (!hasRegisteredCourse)
                            {
                                <li class="nav-item">
                                    <a class="nav-link text-light" asp-controller="CourseRegistration" asp-
action="Register">Register for Course</a>
                                </li>
                            }
                            <li class="nav-item">
                                <a class="nav-link text-light" asp-controller="Profile" asp-action="Index">My
Profile</a>
                            </li>
                        }

                        <!-- ═══ Students List — Admin only ═══ -->
                        @if (isAdmin)
                        {
                            <li class="nav-item">
                                <a class="nav-link text-light" asp-controller="Students" asp-
action="Index">Students</a>
                            </li>
                        }
```

```
                    <!-- ═══ Register/Login or Logout ═══ -->
                    @if (SignInManager.IsSignedIn(User))
                    {
                        <li class="nav-item">
                            <form asp-controller="Account" asp-action="Logout" method="post" class="form-
inline">
                                <button type="submit" class="nav-link btn btn-link text-light">Logout</
button>
                            </form>
                        </li>
                    }
                    else
                    {
                        <li class="nav-item">
                            <a class="nav-link text-light" asp-controller="Account" asp-
action="Register">Register</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link text-light" asp-controller="Account" asp-
action="Login">Login</a>
                        </li>
                    }
                </ul>

                <!-- ═══ Display logged-in user's email in navbar ═══ -->
                @if (SignInManager.IsSignedIn(User) && user != null)
                {
                    <span class="navbar-text text-info small">
                        Logged in as: <strong>@user.Email</strong>
                    </span>
                }
            </div>
        </div>
```

```
    </nav>
</header>

<div class="container">
    <main role="main" class="pb-3 pt-4">
        <!-- ═══ Success / Error messages ═══ -->
        @if (TempData["SuccessMessage"] != null)
        {
            <div class="alert alert-success alert-dismissible fade show" role="alert">
                @TempData["SuccessMessage"]
                <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
            </div>
        }
        @if (TempData["ErrorMessage"] != null)
        {
            <div class="alert alert-danger alert-dismissible fade show" role="alert">
                @TempData["ErrorMessage"]
                <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
            </div>
        }
        @if (TempData["InfoMessage"] != null)
        {
            <div class="alert alert-info alert-dismissible fade show" role="alert">
                @TempData["InfoMessage"]
                <button type="button" class="btn-close" data-bs-dismiss="alert"></button>
            </div>
        }

        @RenderBody()
    </main>
</div>
```

```
    <footer class="border-top footer text-muted">
        <div class="container">
            &copy; @DateTime.Now.Year - Student Registration Web App
        </div>
    </footer>

    <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></script>
    <script src="~/js/site.js" asp-append-version="true"></script>
    @await RenderSectionAsync("Scripts", required: false)
</body>
</html>
```

📄 Views/Shared/Error.cshtml

```
@model ErrorViewModel
@{ ViewData["Title"] = "Error"; }
<h1 class="text-danger">Error.</h1>
<h2 class="text-danger">An error occurred while processing your request.</h2>
@if (Model.ShowRequestId) { <p><strong>Request ID:</strong> <code>@Model.RequestId</code></p> }
```

Views — Home & Account

📄 Views/Home/Index.cshtml

```
@{
    ViewData["Title"] = "Home";
}

<div class="text-center">
    <h1 class="display-4">Welcome to Student Registration Web App</h1>
    <p class="lead">A role-based student course registration system built with ASP.NET Core Identity.</p>
```

```
    <div class="mt-5">
        @if (User.Identity?.IsAuthenticated == true)
        {
            <h3>Hello, @User.Identity?.Name!</h3>
            <p class="text-muted">You are logged in. Use the navigation bar to access your features.</p>
        }
        else
        {
            <div class="row justify-content-center mt-4">
                <div class="col-md-4">
                    <div class="card shadow">
                        <div class="card-body text-center">
                            <h5 class="card-title">New User?</h5>
                            <p class="card-text">Register to create your student account and enroll in
courses.</p>
                            <a asp-controller="Account" asp-action="Register" class="btn btn-primary">Register
Now</a>
                        </div>
                    </div>
                </div>
                <div class="col-md-4">
                    <div class="card shadow">
                        <div class="card-body text-center">
                            <h5 class="card-title">Already Registered?</h5>
                            <p class="card-text">Login to access your profile and course registrations.</p>
                            <a asp-controller="Account" asp-action="Login" class="btn btn-success">Login</a>
                        </div>
                    </div>
                </div>
            </div>
        }
    </div>
```

```
    <div class="mt-5">
        <h4>Available Courses</h4>
        <a asp-controller="Courses" asp-action="Index" class="btn btn-outline-primary">Browse Courses</a>
    </div>
</div>
```

📄 Views/Account/Register.cshtml

```
@model StudentRegistrationWebApp.Controllers.RegisterViewModel
@{ ViewData["Title"] = "Register"; }
<h2>Create a New Account</h2>
<hr />
<div class="row">
    <div class="col-md-6">
        <form asp-action="Register">
            <div asp-validation-summary="All" class="text-danger"></div>
            <div class="mb-3">
                <label asp-for="FullName" class="form-label"></label>
                <input asp-for="FullName" class="form-control" />
                <span asp-validation-for="FullName" class="text-danger"></span>
            </div>
            <div class="mb-3">
                <label asp-for="Email" class="form-label"></label>
                <input asp-for="Email" class="form-control" />
                <span asp-validation-for="Email" class="text-danger"></span>
            </div>
            <div class="mb-3">
                <label asp-for="PhoneNumber" class="form-label"></label>
                <input asp-for="PhoneNumber" class="form-control" />
                <span asp-validation-for="PhoneNumber" class="text-danger"></span>
            </div>
```

```
            <div class="mb-3">
                <label asp-for="Password" class="form-label"></label>
                <input asp-for="Password" class="form-control" type="password" />
                <span asp-validation-for="Password" class="text-danger"></span>
            </div>
            <div class="mb-3">
                <label asp-for="ConfirmPassword" class="form-label"></label>
                <input asp-for="ConfirmPassword" class="form-control" type="password" />
                <span asp-validation-for="ConfirmPassword" class="text-danger"></span>
            </div>
            <div class="mb-3">
                <input type="submit" value="Register" class="btn btn-primary" />
                <a asp-action="Login" class="btn btn-secondary">Already have an account? Login</a>
            </div>
        </form>
    </div>
</div>
```

📄 Views/Account/Login.cshtml

```
@model StudentRegistrationWebApp.Controllers.LoginViewModel
@{ ViewData["Title"] = "Login"; }
<h2>Login</h2>
<hr />
<div class="row">
    <div class="col-md-6">
        <form asp-action="Login">
            <div asp-validation-summary="All" class="text-danger"></div>
            <input type="hidden" name="returnUrl" value='@ViewData["ReturnUrl"]' />
            <div class="mb-3">
                <label asp-for="Email" class="form-label"></label>
                <input asp-for="Email" class="form-control" />
```

```
                <span asp-validation-for="Email" class="text-danger"></span>
            </div>
            <div class="mb-3">
                <label asp-for="Password" class="form-label"></label>
                <input asp-for="Password" class="form-control" type="password" />
                <span asp-validation-for="Password" class="text-danger"></span>
            </div>
            <div class="mb-3 form-check">
                <input asp-for="RememberMe" class="form-check-input" />
                <label asp-for="RememberMe" class="form-check-label"></label>
            </div>
            <div class="mb-3">
                <input type="submit" value="Login" class="btn btn-success" />
                <a asp-action="Register" class="btn btn-secondary">New user? Register</a>
            </div>
        </form>
    </div>
</div>
```

📄 Views/Account/AccessDenied.cshtml

```
@{ ViewData["Title"] = "Access Denied"; }
<div class="text-center mt-5">
    <h1 class="text-danger display-4">⛔ Access Denied</h1>
    <p class="lead mt-3">You do not have permission to access this page.</p>
    <p class="text-muted">If you believe this is an error, please contact the administrator.</p>
    <a asp-controller="Home" asp-action="Index" class="btn btn-primary mt-3">Back to Home</a>
</div>
```

Views — Courses

📄 Views/Courses/Index.cshtml

```
@model IEnumerable<StudentRegistrationWebApp.Models.Course>

@{
    ViewData["Title"] = "Courses";
}

<div class="d-flex justify-content-between align-items-center mb-3">
    <h2>Available Courses</h2>
    @if (User.IsInRole("Admin"))
    {
        <a asp-action="Create" class="btn btn-primary">+ Add New Course</a>
    }
</div>

<div class="table-responsive">
    <table class="table table-striped table-hover">
        <thead class="table-dark">
            <tr>
                <th>@Html.DisplayNameFor(model => model.First().CourseCode)</th>
                <th>@Html.DisplayNameFor(model => model.First().CourseName)</th>
                <th>@Html.DisplayNameFor(model => model.First().Credits)</th>
                <th>@Html.DisplayNameFor(model => model.First().Instructor)</th>
                <th>Actions</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var item in Model)
            {
                <tr>
                    <td><strong>@Html.DisplayFor(modelItem => item.CourseCode)</strong></td>
                    <td>@Html.DisplayFor(modelItem => item.CourseName)</td>
```

```
                    <td>@Html.DisplayFor(modelItem => item.Credits)</td>
                    <td>@Html.DisplayFor(modelItem => item.Instructor)</td>
                    <td>
                        <a asp-action="Details" asp-route-id="@item.CourseId" class="btn btn-sm btn-
info">Details</a>
                        @if (User.IsInRole("Admin"))
                        {
                            <a asp-action="Edit" asp-route-id="@item.CourseId" class="btn btn-sm btn-
warning">Edit</a>
                            <a asp-action="Delete" asp-route-id="@item.CourseId" class="btn btn-sm btn-
danger">Delete</a>
                        }
                        @if (User.IsInRole("Student"))
                        {
                            <form asp-controller="CourseRegistration" asp-action="Register" asp-route-
courseId="@item.CourseId" method="post" class="d-inline">
                                <button type="submit" class="btn btn-sm btn-success">Register</button>
                            </form>
                        }
                    </td>
                </tr>
            }
        </tbody>
    </table>
</div>

@if (!Model.Any())
{
    <div class="alert alert-info text-center">No courses available yet.</div>
}
```

📄 Views/Courses/Details.cshtml

```
@model StudentRegistrationWebApp.Models.Course
```

```
@{ ViewData["Title"] = "Course Details"; }

<h2>Course Details</h2>

<hr />

<div class="card" style="max-width: 600px;">

    <div class="card-body">

        <h4 class="card-title">@Model.CourseName</h4>

        <p class="text-muted"><strong>Code:</strong> @Model.CourseCode</p>

        <p><strong>Description:</strong> @Model.Description</p>

        <p><strong>Credits:</strong> @Model.Credits</p>

        <p><strong>Instructor:</strong> @Model.Instructor</p>

    </div>

</div>

<div class="mt-3">

    @if (User.IsInRole("Admin"))

    {

        <a asp-action="Edit" asp-route-id="@Model.CourseId" class="btn btn-warning">Edit</a>

        <a asp-action="Delete" asp-route-id="@Model.CourseId" class="btn btn-danger">Delete</a>

    }

    <a asp-action="Index" class="btn btn-secondary">Back to List</a>

</div>
```

📄 Views/Courses/Create.cshtml

```
@model StudentRegistrationWebApp.Models.Course

@{ ViewData["Title"] = "Create Course"; }

<h2>Create New Course</h2>

<hr />

<div class="row">

    <div class="col-md-6">

        <form asp-action="Create">

            <div asp-validation-summary="ModelOnly" class="text-danger"></div>

            <div class="mb-3">
```

```
            <label asp-for="CourseName" class="form-label"></label>
            <input asp-for="CourseName" class="form-control" />
            <span asp-validation-for="CourseName" class="text-danger"></span>
        </div>
        <div class="mb-3">
            <label asp-for="CourseCode" class="form-label"></label>
            <input asp-for="CourseCode" class="form-control" />
            <span asp-validation-for="CourseCode" class="text-danger"></span>
        </div>
        <div class="mb-3">
            <label asp-for="Description" class="form-label"></label>
            <textarea asp-for="Description" class="form-control" rows="3"></textarea>
            <span asp-validation-for="Description" class="text-danger"></span>
        </div>
        <div class="mb-3">
            <label asp-for="Credits" class="form-label"></label>
            <input asp-for="Credits" class="form-control" type="number" />
            <span asp-validation-for="Credits" class="text-danger"></span>
        </div>
        <div class="mb-3">
            <label asp-for="Instructor" class="form-label"></label>
            <input asp-for="Instructor" class="form-control" />
            <span asp-validation-for="Instructor" class="text-danger"></span>
        </div>
        <div class="mb-3">
            <input type="submit" value="Create" class="btn btn-primary" />
            <a asp-action="Index" class="btn btn-secondary">Back to List</a>
        </div>
    </form>
  </div>
</div>
```

Views/Courses/Edit.cshtml

```
@model StudentRegistrationWebApp.Models.Course

@{ ViewData["Title"] = "Edit Course"; }

<h2>Edit Course</h2>

<hr />

<div class="row">

    <div class="col-md-6">

        <form asp-action="Edit">

            <div asp-validation-summary="ModelOnly" class="text-danger"></div>

            <input type="hidden" asp-for="CourseId" />

            <div class="mb-3">

                <label asp-for="CourseName" class="form-label"></label>

                <input asp-for="CourseName" class="form-control" />

                <span asp-validation-for="CourseName" class="text-danger"></span>

            </div>

            <div class="mb-3">

                <label asp-for="CourseCode" class="form-label"></label>

                <input asp-for="CourseCode" class="form-control" />

                <span asp-validation-for="CourseCode" class="text-danger"></span>

            </div>

            <div class="mb-3">

                <label asp-for="Description" class="form-label"></label>

                <textarea asp-for="Description" class="form-control" rows="3"></textarea>

                <span asp-validation-for="Description" class="text-danger"></span>

            </div>

            <div class="mb-3">

                <label asp-for="Credits" class="form-label"></label>

                <input asp-for="Credits" class="form-control" type="number" />

                <span asp-validation-for="Credits" class="text-danger"></span>

            </div>
```

```
            <div class="mb-3">
                <label asp-for="Instructor" class="form-label"></label>
                <input asp-for="Instructor" class="form-control" />
                <span asp-validation-for="Instructor" class="text-danger"></span>
            </div>
            <div class="mb-3">
                <input type="submit" value="Save" class="btn btn-warning" />
                <a asp-action="Index" class="btn btn-secondary">Back to List</a>
            </div>
        </form>
    </div>
</div>
```

📄 Views/Courses/Delete.cshtml

```
@model StudentRegistrationWebApp.Models.Course
@{ ViewData["Title"] = "Delete Course"; }
<h2>Delete Course</h2>
<hr />
<div class="alert alert-danger">
    <h5>Are you sure you want to delete this course?</h5>
</div>
<div class="card" style="max-width: 600px;">
    <div class="card-body">
        <h4>@Model.CourseName</h4>
        <p><strong>Code:</strong> @Model.CourseCode</p>
        <p><strong>Credits:</strong> @Model.Credits</p>
        <p><strong>Instructor:</strong> @Model.Instructor</p>
    </div>
</div>
<form asp-action="Delete" class="mt-3">
    <input type="hidden" asp-for="CourseId" />
```

```
    <input type="submit" value="Delete" class="btn btn-danger" />
    <a asp-action="Index" class="btn btn-secondary">Cancel</a>
</form>
```

Views — Students, Profile & Course Registration

📄 Views/Students/Index.cshtml

```
@model IEnumerable<StudentRegistrationWebApp.Controllers.StudentViewModel>
@{ ViewData["Title"] = "Registered Students"; }
<h2>Registered Students</h2>
<hr />
<table class="table table-striped table-hover">
    <thead class="table-dark">
        <tr>
            <th>Full Name</th>
            <th>Email</th>
            <th>Phone</th>
            <th>Registered Course</th>
            <th>Registered On</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var s in Model)
        {
            <tr>
                <td>@s.FullName</td>
                <td>@s.Email</td>
                <td>@(string.IsNullOrEmpty(s.PhoneNumber) ? "N/A" : s.PhoneNumber)</td>
                <td><span class="badge bg-success">@s.RegisteredCourse</span></td>
                <td>@s.RegisteredOn.ToString("yyyy-MM-dd")</td>
            </tr>
        }
```

```
    </tbody>
</table>
@if (!Model.Any())
{
    <div class="alert alert-info text-center">No students registered yet.</div>
}
```

📄 Views/Profile/Index.cshtml

```
@model StudentRegistrationWebApp.Controllers.ProfileViewModel
@{ ViewData["Title"] = "My Profile"; }
<h2>My Profile</h2>
<hr />
<div class="card" style="max-width: 600px;">
    <div class="card-body">
        <h4 class="card-title">@Model.FullName</h4>
        <p><strong>Email:</strong> @Model.Email</p>
        <p><strong>Phone:</strong> @(string.IsNullOrEmpty(Model.PhoneNumber) ? "N/A" : Model.PhoneNumber)</p>
        <p><strong>Address:</strong> @(string.IsNullOrEmpty(Model.Address) ? "N/A" : Model.Address)</p>
        <p><strong>Registered On:</strong> @Model.RegisteredOn.ToString("yyyy-MM-dd")</p>
        <hr />
        <p><strong>Registered Course:</strong>
            @if (Model.RegisteredCourse != "Not Registered")
            {
                <span class="badge bg-success">@Model.RegisteredCourse (@Model.CourseCode)</span>
            }
            else
            {
                <span class="badge bg-warning text-dark">Not Registered</span>
                <a asp-controller="CourseRegistration" asp-action="Register" class="btn btn-sm btn-primary
ms-2">Register for a Course</a>
            }
```

```
        </p>
    </div>
</div>
<div class="mt-3">
    <a asp-action="Edit" class="btn btn-warning">Edit Profile</a>
</div>
```

📄 Views/Profile/Edit.cshtml

```
@model StudentRegistrationWebApp.Controllers.EditProfileViewModel
@{ ViewData["Title"] = "Edit Profile"; }
<h2>Edit Profile</h2>
<hr />
<div class="row">
    <div class="col-md-6">
        <form asp-action="Edit">
            <div asp-validation-summary="ModelOnly" class="text-danger"></div>
            <div class="mb-3">
                <label asp-for="FullName" class="form-label"></label>
                <input asp-for="FullName" class="form-control" />
                <span asp-validation-for="FullName" class="text-danger"></span>
            </div>
            <div class="mb-3">
                <label asp-for="PhoneNumber" class="form-label"></label>
                <input asp-for="PhoneNumber" class="form-control" />
                <span asp-validation-for="PhoneNumber" class="text-danger"></span>
            </div>
            <div class="mb-3">
                <label asp-for="Address" class="form-label"></label>
                <input asp-for="Address" class="form-control" />
                <span asp-validation-for="Address" class="text-danger"></span>
            </div>
```

```
            <div class="mb-3">
                <input type="submit" value="Save Changes" class="btn btn-warning" />
                <a asp-action="Index" class="btn btn-secondary">Back to Profile</a>
            </div>
        </form>
    </div>
</div>
```

📄 Views/CourseRegistration/Register.cshtml

```
@model IEnumerable<StudentRegistrationWebApp.Models.Course>
@{ ViewData["Title"] = "Register for a Course"; }
<h2>Register for a Course</h2>
<p class="text-muted">Students can register for <strong>only one</strong> course. Choose carefully.</p>
<hr />
<table class="table table-striped">
    <thead class="table-dark">
        <tr>
            <th>Code</th>
            <th>Course Name</th>
            <th>Credits</th>
            <th>Instructor</th>
            <th>Action</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var c in Model)
        {
            <tr>
                <td><strong>@c.CourseCode</strong></td>
                <td>@c.CourseName</td>
                <td>@c.Credits</td>
```

```
                <td>@c.Instructor</td>

                <td>

                    <form asp-action="Register" asp-route-courseId="@c.CourseId" method="post" class="d-
inline">

                        <button type="submit" class="btn btn-sm btn-success" onclick="return confirm('Register
for @c.CourseName?')">Register</button>

                    </form>

                </td>

            </tr>

        }

    </tbody>

</table>
```